

Introduction to Transformers

Naeemullah Khan

naeemullah.khan@kaust.edu.sa

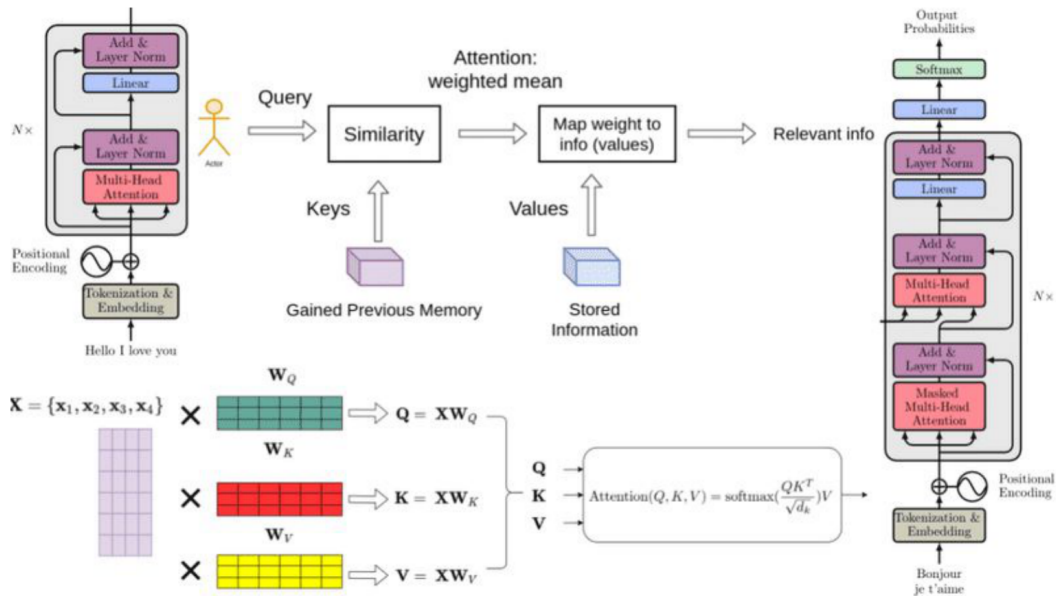


جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

July 31, 2026



1. Motivation
2. Learning Outcomes
3. Introduction to Transformers
4. Self-Attention Mechanism
5. Multi-Head Attention
6. Positional Encoding
7. Transformer Block
8. Encoder-Decoder Architecture
9. Causal Masking
10. Pre-LayerNorm vs. Post-LayerNorm Architectures
11. Layer Normalization vs RMSNorm
12. Transformer Variants
13. Limitations of Transformers
14. Summary

► **Traditional RNNs/LSTMs suffer from:**

- Sequential computation → slow training
- Vanishing gradients → poor long-range dependencies

► **Need for a model that:**

- Handles global context
- Is parallelizable
- Supports scalability

After this session, you should be able to:

- ▶ Understand the self-attention mechanism
- ▶ Learn how multi-head attention works and why it helps
- ▶ Grasp the role of positional encoding
- ▶ Deconstruct transformer blocks and variants
- ▶ Compare Pre-LayerNorm vs. Post-LayerNorm
- ▶ Evaluate LayerNorm vs. RMSNorm
- ▶ Explore causal masking in decoder-only transformers
- ▶ Understand tradeoffs and future directions

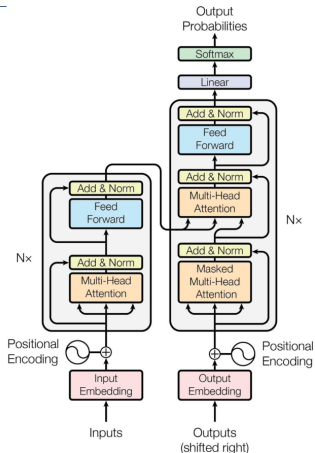
► What is a Transformer?

- Introduced by Vaswani et al. (2017) – “Attention is All You Need”

► Key features:

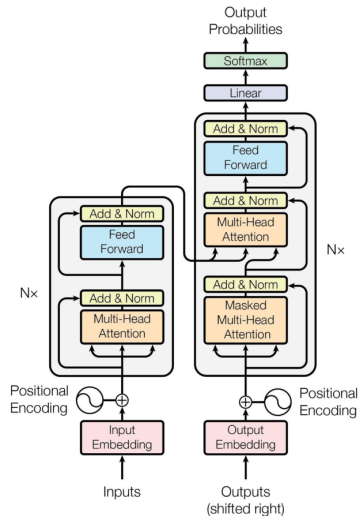
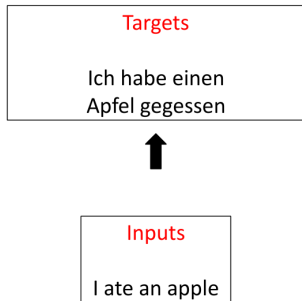
- No recurrence, all attention
- Encoder-decoder structure
- Enables parallel processing
- Scales well with data and compute

- ▶ Tokenization
- ▶ Input Embeddings
- ▶ Position Encodings
- ▶ Query, Key, & Value
- ▶ Attention
- ▶ Self Attention
- ▶ Multi-Head Attention
- ▶ Feed Forward
- ▶ Add & Norm
- ▶ Encoders
- ▶ Masked Attention
- ▶ Encoder Decoder Attention
- ▶ Linear
- ▶ Softmax
- ▶ Decoders
- ▶ Encoder-Decoder Models



Source: Vaswani et al., "Attention Is All You Need,"

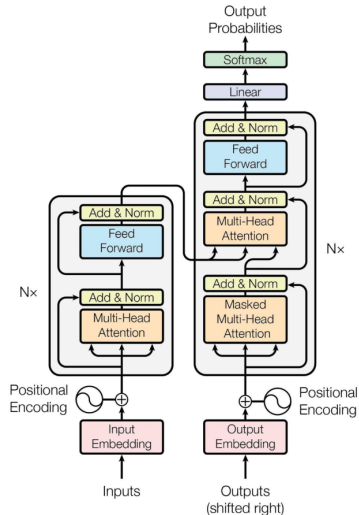
NeurIPS 2017, Fig. 1.



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 7.

Processing Inputs

Inputs
I ate an apple



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 8.

- ▶ Computes contextual representation of a word using all other words
- ▶ **Inputs:** Query (Q), Key (K), Value (V)
- ▶ **Output:** Weighted sum of values

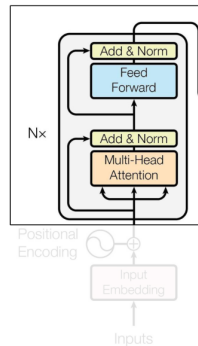
Attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- ▶ Enables each word to attend to all others

- ▶ **Query:** What we're looking for (e.g., the meaning of “bank” in context)
- ▶ **Keys:** Tags or features associated with each word/sentence (e.g., “river”, “money”)
- ▶ **Values:** Information linked to each key (e.g., context-specific meaning)
- ▶ Uses dot-product similarity between Query and Keys to assign attention weights
- ▶ Weighted sum of Values produces the contextual representation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



The

animal

didn't

cross

the

street

because

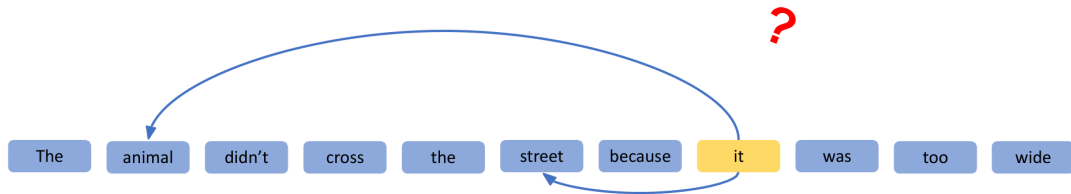
it

was

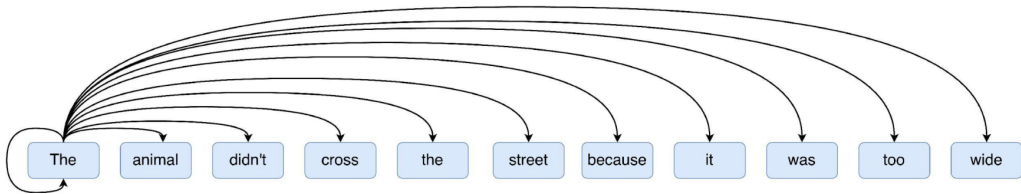
too

wide

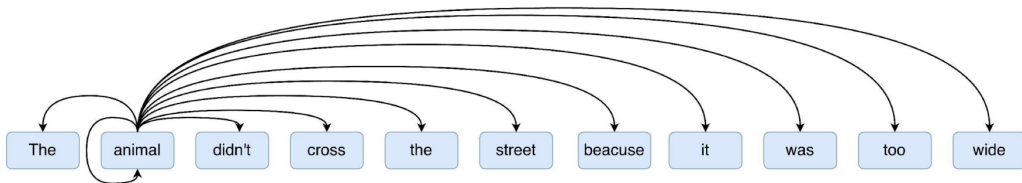
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 61.



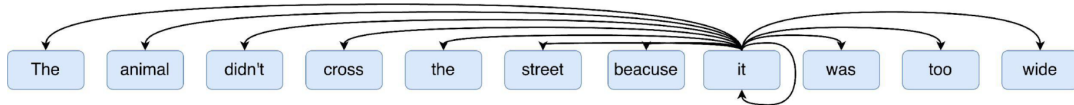
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 62.



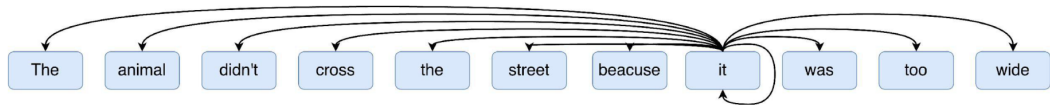
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 63.



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 64.



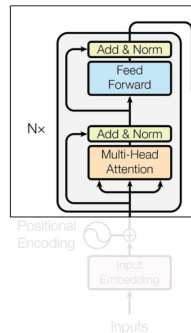
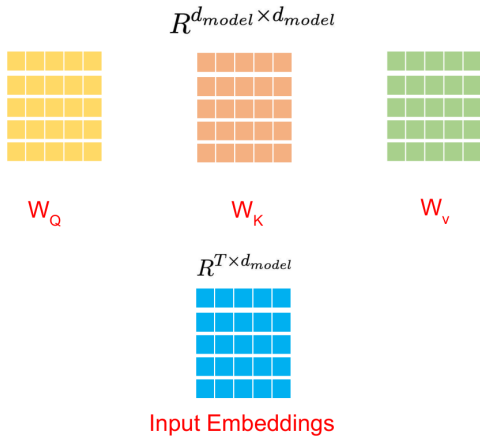
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 65.



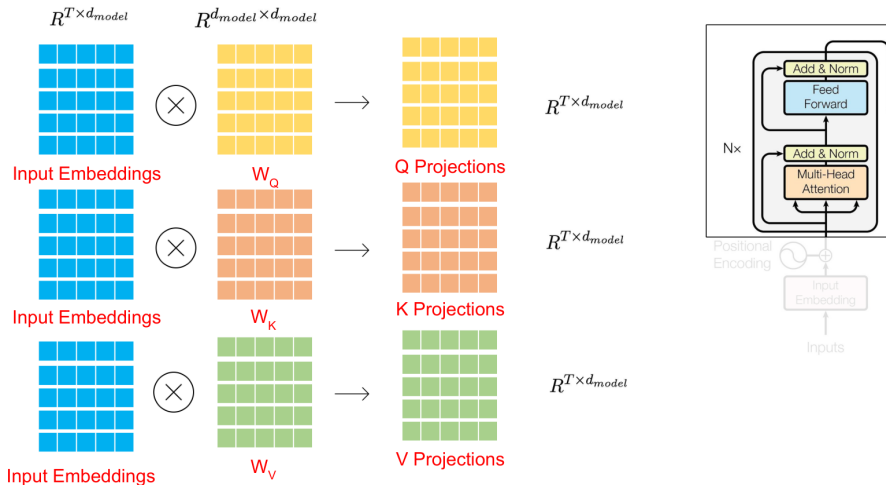
SELF

Query Inputs = Key Inputs = Value Inputs

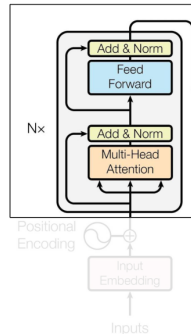
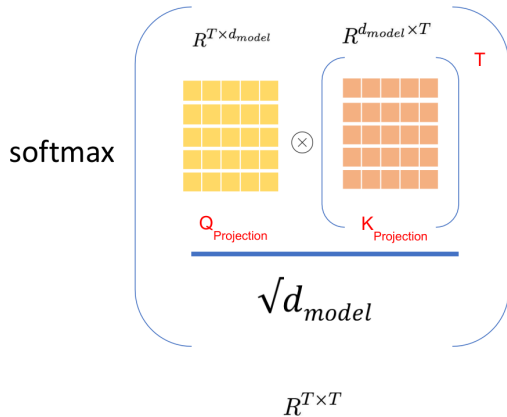
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 66.



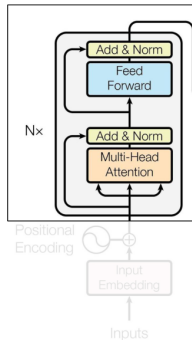
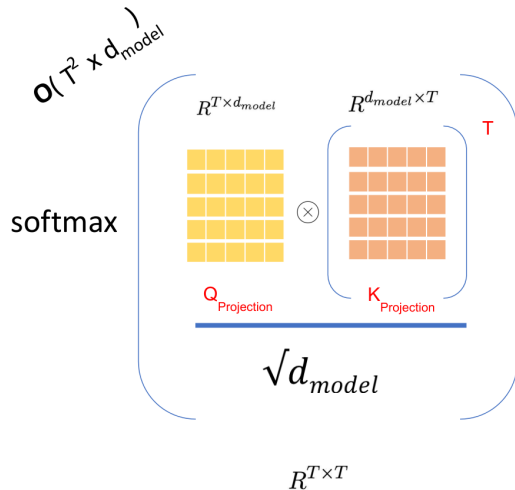
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 67.



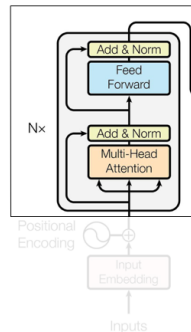
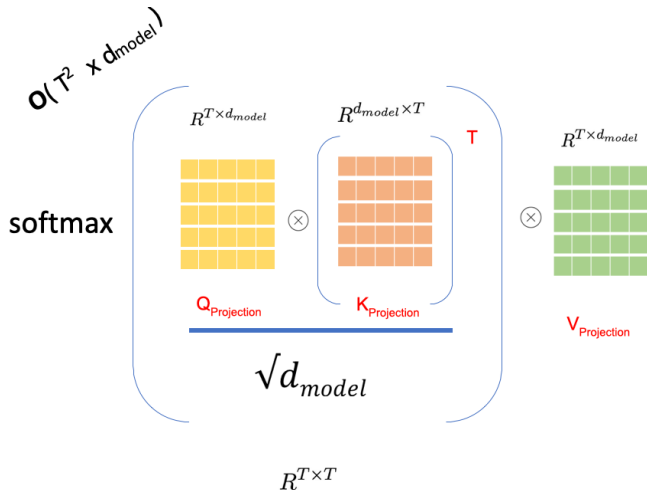
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 68.



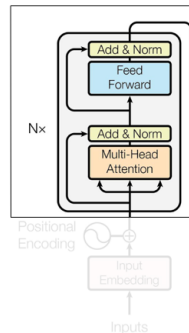
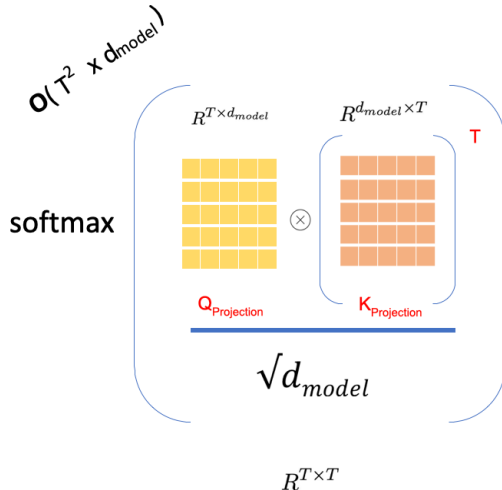
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 69.



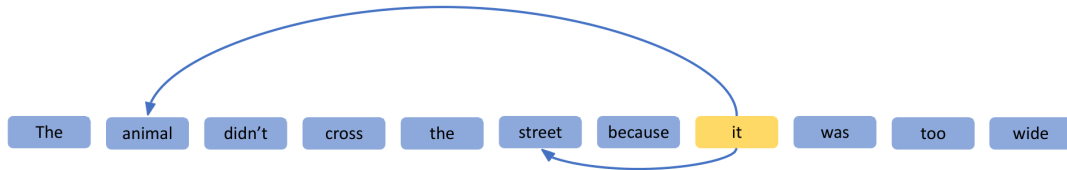
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 70.



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 71.

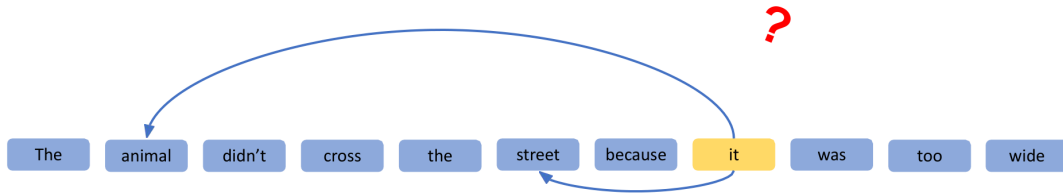


Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 72.



Coreference resolution ✓

Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 73.



Sentence boundaries ?

Coreference resolution



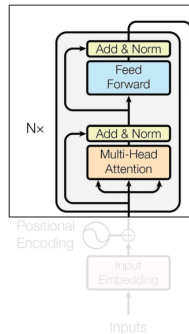
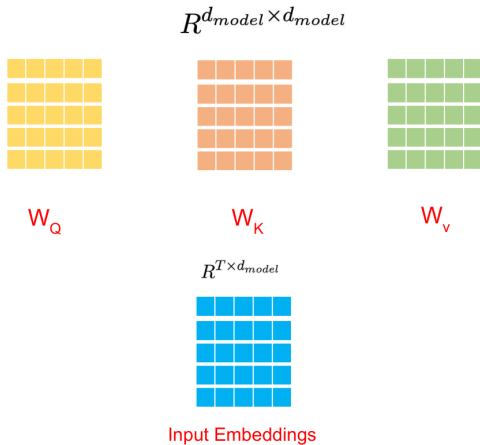
Context ?

Semantic relationships ?

Part of Speech ?

Comparisons ?

Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 74.



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 75.

- ▶ Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions.
- ▶ It consists of multiple attention heads, each with its own set of parameters.
- ▶ The outputs of all heads are concatenated and linearly transformed.

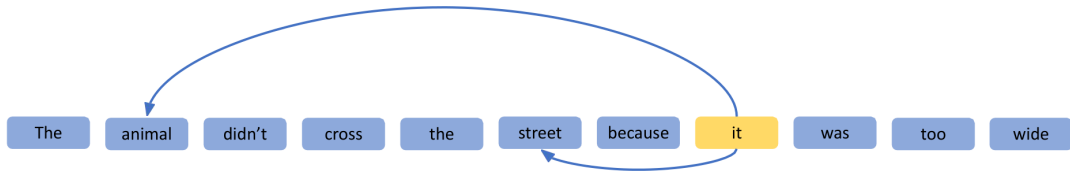
Formula:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where each head is defined as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- ▶ W_i^Q , W_i^K , and W_i^V are learned projection matrices for each head.
- ▶ W^O is the output projection matrix that combines the outputs of all heads.



Sentence boundaries ✓

Coreference resolution ✓

Context ✓

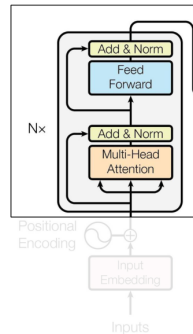
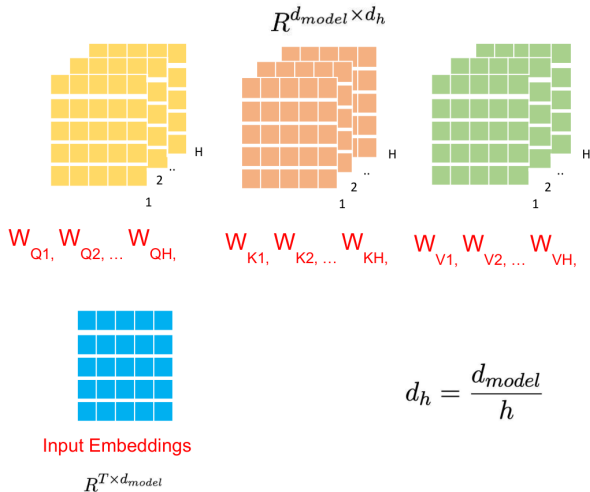
Semantic relationships ✓

Part of speech ✓

Comparisons ✓

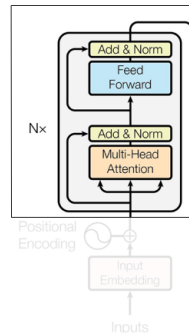
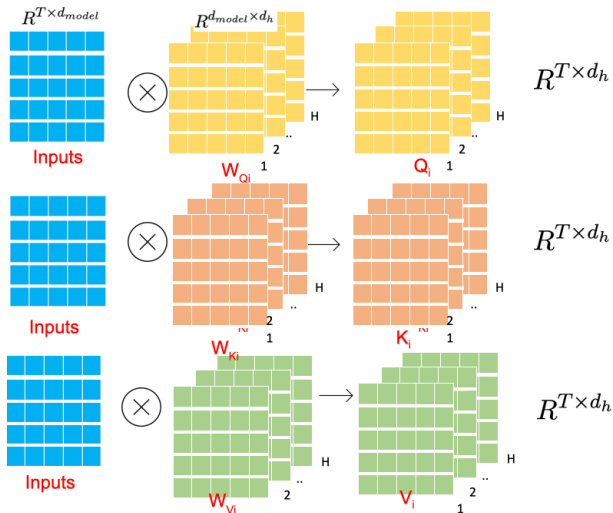
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 80.

Multi-Head Attention

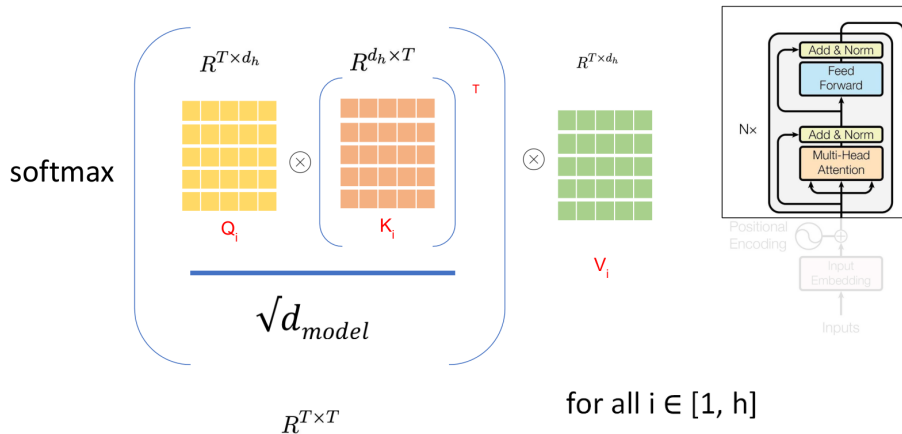


Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 76.

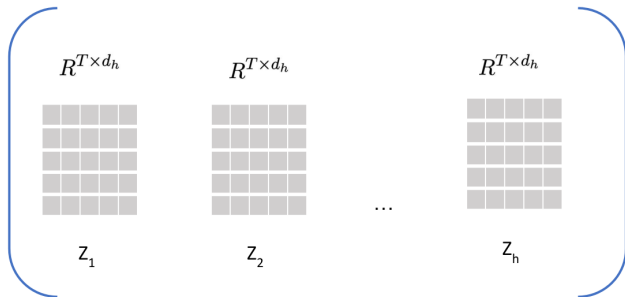
Multi-Head Attention



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 77.



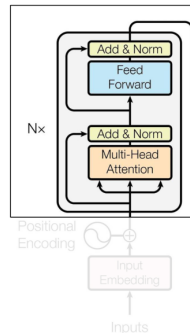
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 78.



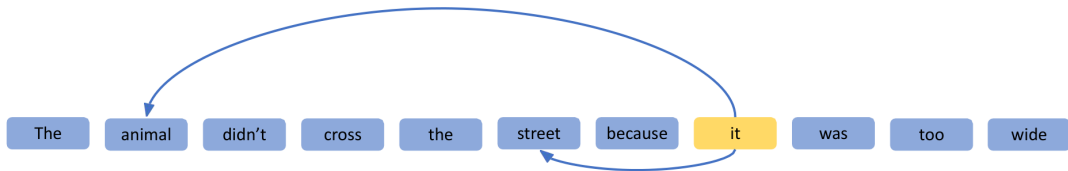
Multi Head Attention : Z

$$d_h = \frac{d_{model}}{h}$$

$$R^{T \times d_{model}}$$



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 79.



Sentence boundaries	✓	Coreference resolution	✓	Context	✓
Semantic relationships	✓	Part of speech	✓	Comparisons	✓

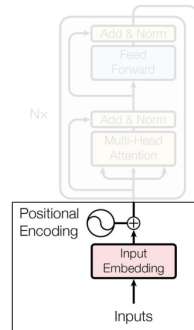
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 80.

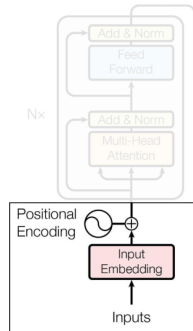
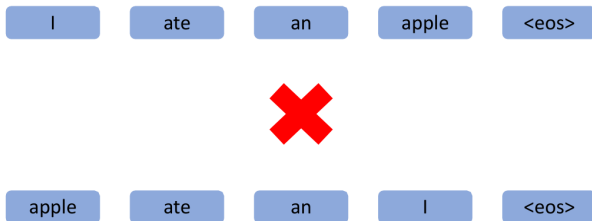
- ▶ Enables the model to jointly attend to information from different representation subspaces at different positions.
- ▶ Captures various linguistic and syntactic patterns by using multiple attention heads.
- ▶ Improves the model's ability to represent complex relationships in the data.
- ▶ Provides richer and more diverse feature representations.
- ▶ Costs no more than single-head attention: each head uses $d_k = d_v = d_{\text{model}}/h$, so the h heads together take the same width as one full-size head.

- ▶ Transformers lack sequence order awareness.
- ▶ **Positional encodings** are added to input embeddings to provide information about token positions.
- ▶ **Sinusoidal encoding:**

$$PE(pos, 2i) = \sin \left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}} \right)$$
$$PE(pos, 2i + 1) = \cos \left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}} \right)$$

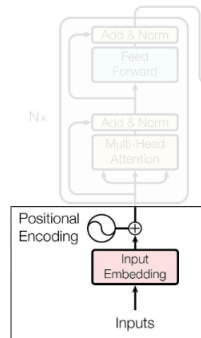
I ate an apple <eos>





Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 14.

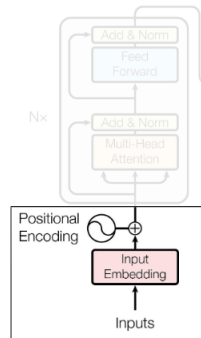
Requirements for Positional Encodings???



Source: CMU 11-785, Lecture 19, Spring 2024.

Requirements for Positional Encodings

- ▶ Some representation of time? (like seq2seq?)
- ▶ Should be unique for each position – not cyclic



Source: CMU 11-785, Lecture 19, Spring 2024.

Requirements for Positional Encodings

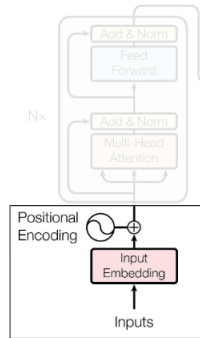
- ▶ Some representation of time? (like seq2seq?)
- ▶ Should be unique for each position – not cyclic

Possible Candidates:

$$P_{t+1} = P_t + \Delta c$$

$$P_{t+1} = e^{t\Delta c}$$

$$P_{t+1} = P_t e^{\Delta c}$$



Source: CMU 11-785, Lecture 19, Spring 2024.

Requirements for Positional Encodings

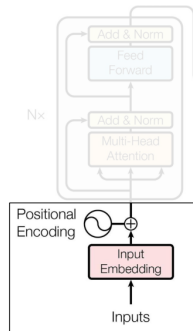
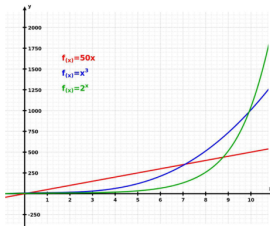
- Some representation of time? (like **seq2seq**?)
- Should be unique for each position – not cyclic

Possible Candidates :

$$P_{t+1} = P_t + \Delta c$$

$$P_{t+1} = e^{P_t \Delta c}$$

$$P_{t+1} = P_t \cdot t \Delta c$$



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 18.

Requirements for Positional Encodings

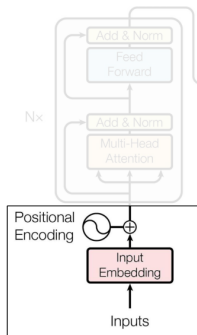
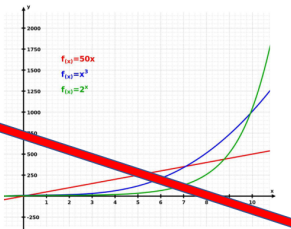
- Some representation of time? (like **seq2seq**?)
- Should be unique for each position – not cyclic

Possible Candidates :

$$P_{t+1} = P_t + \Delta c$$

$$P_{t+1} = P_t \cdot c$$

$$P_{t+1} = P_t \cdot t \Delta c$$



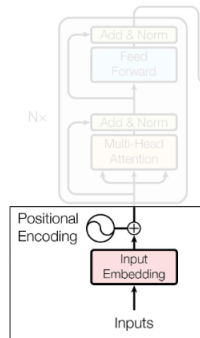
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 19.

Requirements for Positional Encodings

- ▶ Some representation of time? (like seq2seq?)
- ▶ Should be unique for each position – not cyclic
- ▶ **Bounded**

Possible Candidates

$$P(t + t') = M^{t'} \times P(t)$$



Source: CMU 11-785, Lecture 19, Spring 2024.

Requirements for Positional Encodings

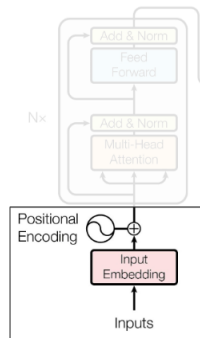
- ▶ Some representation of time? (like seq2seq?)
- ▶ Should be unique for each position – not cyclic
- ▶ **Bounded**

Possible Candidates

$$P(t + t') = M^{t'} \times P(t)$$

M?

1. Should be a unitary matrix
2. Magnitudes of eigenvalues should be 1 $\rightarrow$ norm preserving
3. The matrix can be learnt
4. Produces unique rotated embeddings each time



Source: CMU 11-785, Lecture 19, Spring 2024.

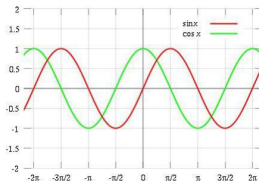
Requirements for Position Encodings

- Some representation of time? (like **seq2seq**?)
- Should be unique for each position
- Bounded

Actual Candidates

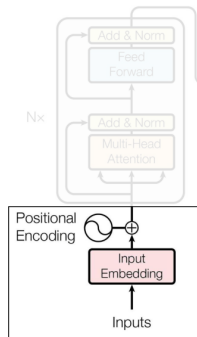
$\text{sine}(g(t))$

$\text{cosine}(g(t))$



Requirements for $g(t)$

- Must have same dimensions as input embeddings
- Must produce overall unique encodings



For each position, an embedded input is moved the same distance but at a different angle. **Inputs that are close to each other in the sequence have similar perturbations, but inputs that are far apart are perturbed in different directions.**

$$PE_{(pos, 2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right)$$

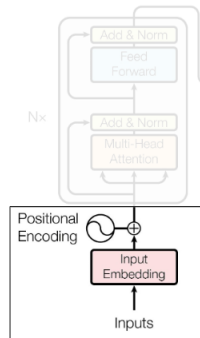
pos idx of the token in input sentence

i i^{th} dimension out of d_{model}

d_{model} embedding dimension of each token

Different calculations for odd and even embedding indices.

Each dimension is periodic; the full vector is not.



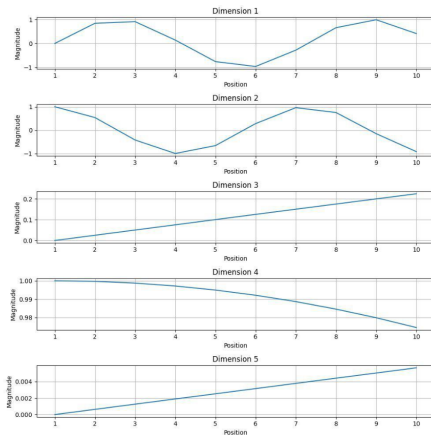
Source: CMU 11-785, Lecture 19, Spring 2024.



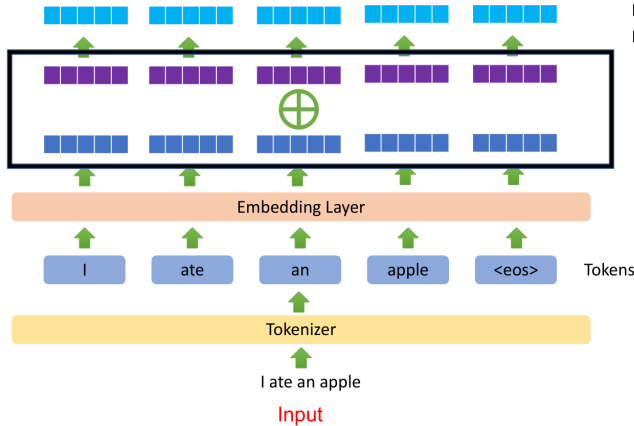
Positional Encoding:

	0	1	2	3	4
Dim 1	0.000	0.841	0.909	0.141	-0.757
Dim 2	1.000	0.540	-0.416	-0.990	-0.654
Dim 3	0.000	0.025	0.050	0.075	0.100
Dim 4	1.000	1.000	0.999	0.997	0.995
Dim 5	0.000	0.001	0.001	0.002	0.003

Positional Encoding

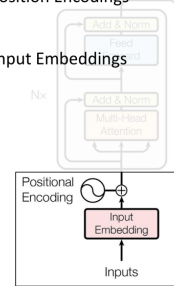


Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 25.



Final Input Embeddings
Position Encodings

Input Embeddings



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 26.

▶ Learnable position embeddings:

- Position embeddings are parameters learned during training.
- Allow the model to adapt position representations to the task.

▶ Rotary Position Embeddings (RoPE):

- Encodes positions by rotating query and key vectors in multi-head attention.
- Enables better extrapolation to longer sequences.

▶ ALiBi (Attention with Linear Biases):

- Adds linear biases to attention scores based on distance between tokens.
- Encourages attention to nearby tokens without explicit position embeddings.

- ▶ **Multi-head Attention**
- ▶ **Add & Norm**
- ▶ **Feed-forward Layer**
- ▶ **Add & Norm**

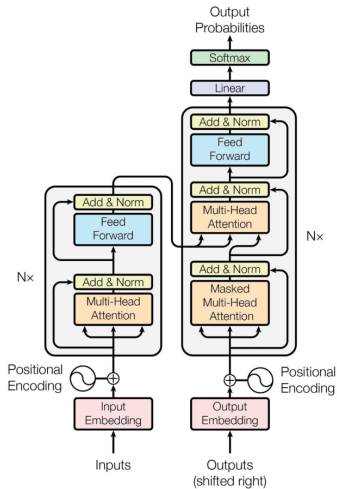
Note

Residual connections and layer normalization are crucial for effective training.

- ▶ The feed-forward sublayer is applied to each position separately, with the same weights at every position:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

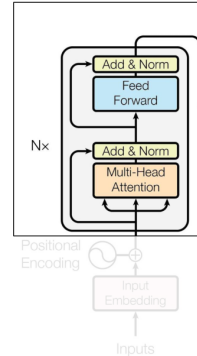
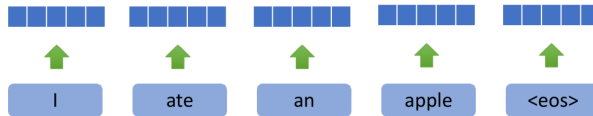
- ▶ It widens the representation to d_{ff} and projects it back, mixing features within a token, whereas attention mixes information across tokens.
- ▶ **Original base model:** 6 encoder blocks and 6 decoder blocks, $d_{\text{model}} = 512$, $h = 8$ heads with $d_k = d_v = 64$, and $d_{\text{ff}} = 2048$.
- ▶ At those sizes the feed-forward sublayer holds $2d_{\text{model}}d_{\text{ff}} \approx 2.1\text{M}$ weights against $4d_{\text{model}}^2 \approx 1.0\text{M}$ for the attention projections: two thirds of the block.



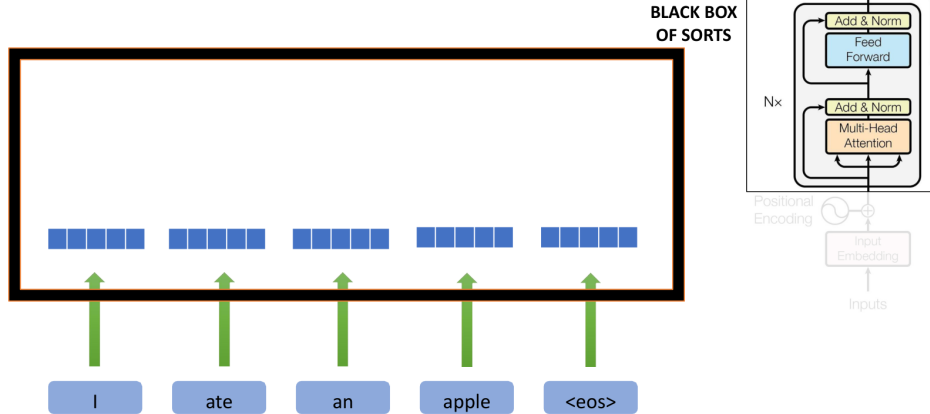
Source: Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1.

- ▶ **Encoder:** Stack of self-attention and feed-forward (FF) layers.
- ▶ **Decoder:** Adds masked self-attention and encoder-decoder attention on top of FF layers.
- ▶ **Applications:** Widely used in tasks like translation, summarization, and more.

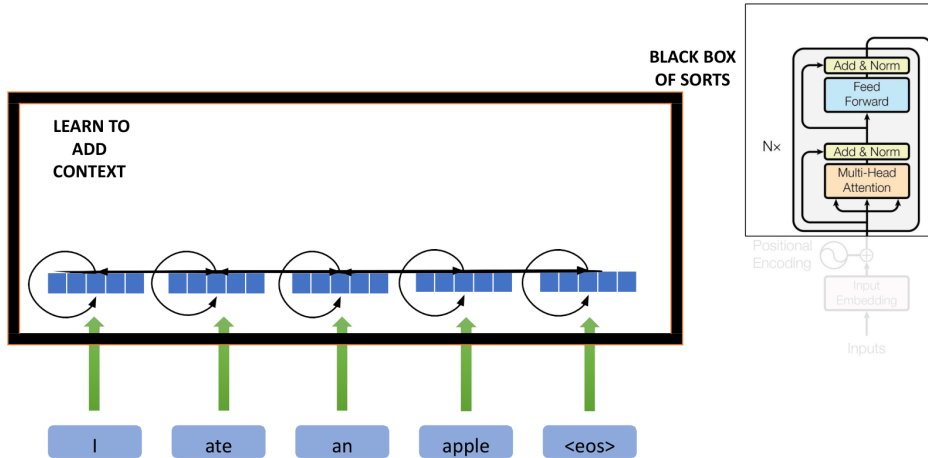
WHERE IS THE
CONTEXT ?



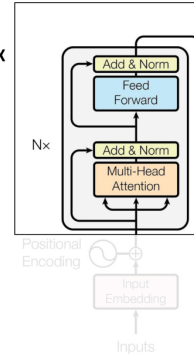
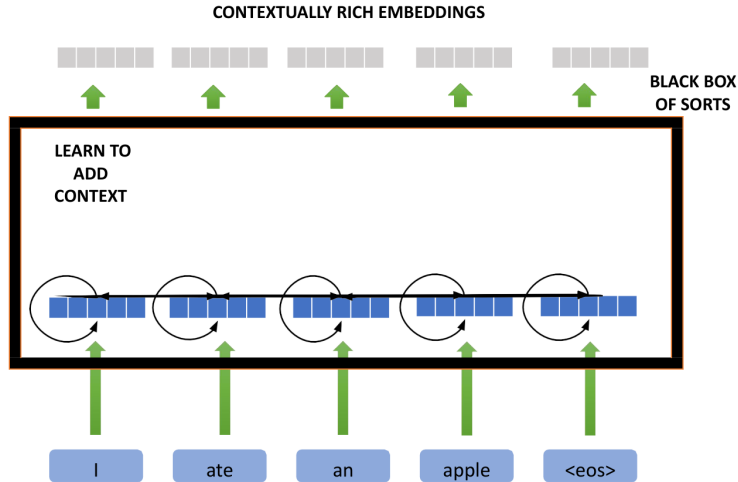
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 28.



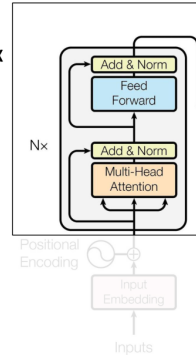
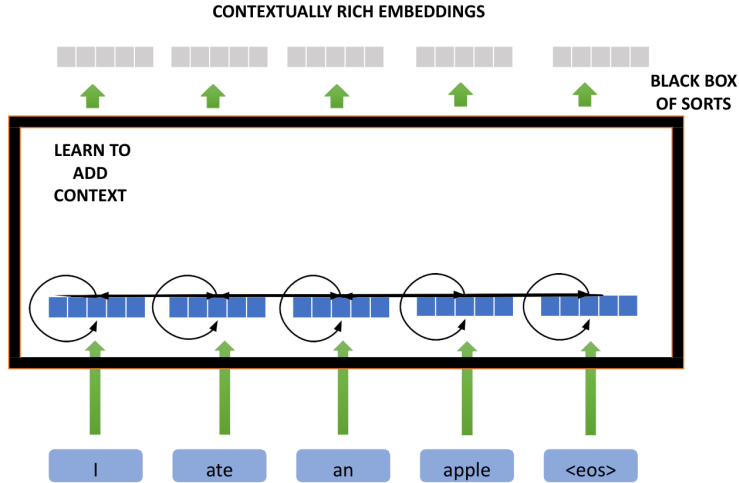
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 29.



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 30.



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 31.

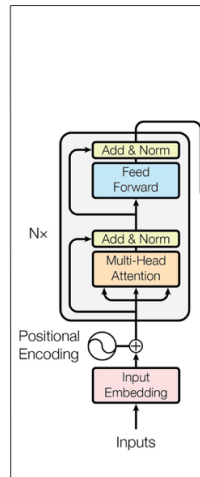


Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 32.

- ▶ **Example:** BERT
- ▶ Only the encoder stack is used.
- ▶ **Applications:** Classification, question answering, embeddings.

Encoder

ENCODER



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 84.

Encoder



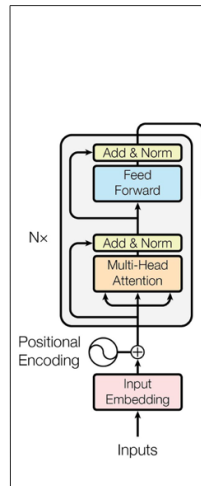
⋮



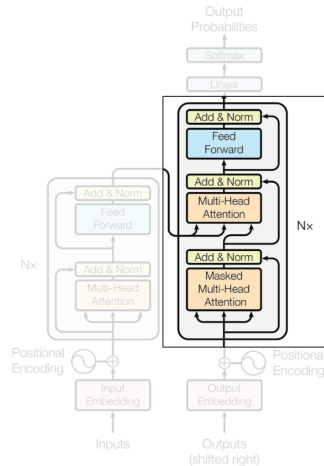
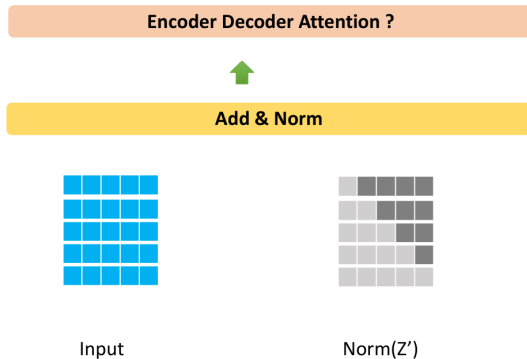
Input to Encoder_{*i*+1}

↑

Output from Encoder_{*i*}

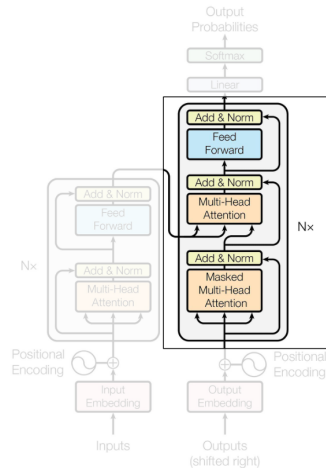


Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 85.



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 101.

Encoder Decoder Attention ?



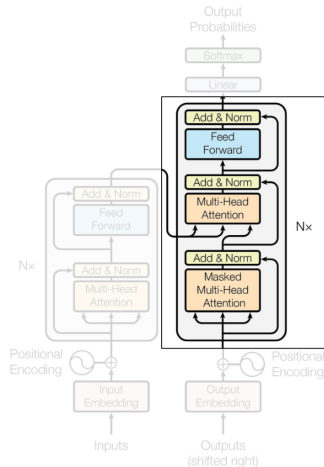
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 102.

Encoder Self Attention

1. Queries from Encoder Inputs
2. Keys from Encoder Inputs
3. Values from Encoder Inputs

Decoder Masked Self Attention

1. Queries from Decoder Inputs
2. Keys from Decoder Inputs
3. Values from Decoder Inputs



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 103.

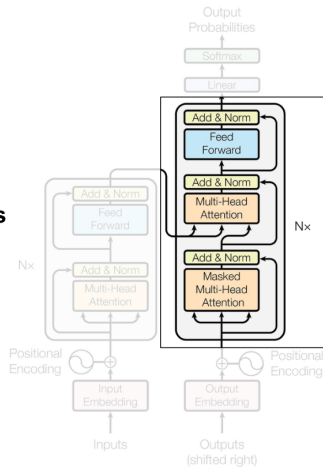
Encoder

Keys from **Encoder Outputs**
Values from **Encoder Outputs**

Decoder

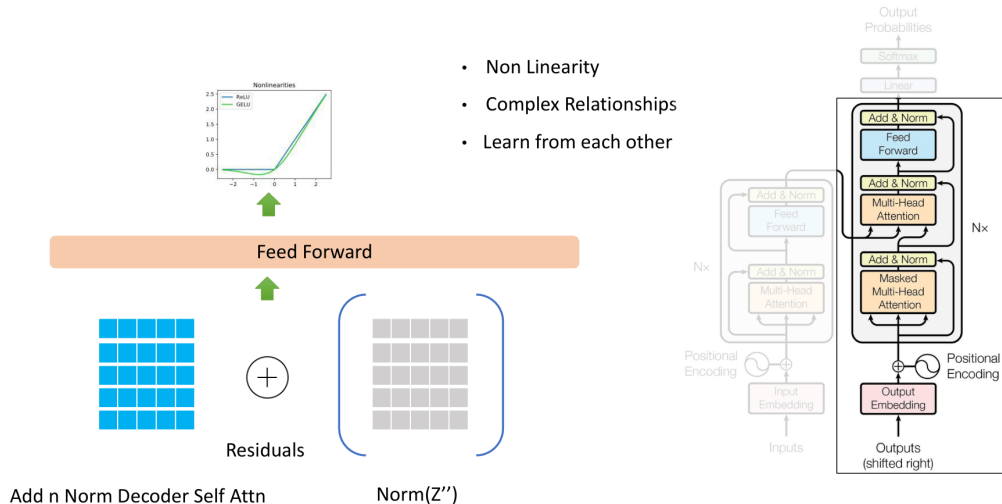
Queries from **Decoder Inputs**

NOTE: Every decoder block receives the same FINAL encoder output



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 105.

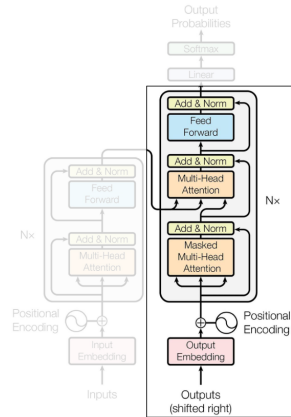
- Non Linearity
- Complex Relationships
- Learn from each other



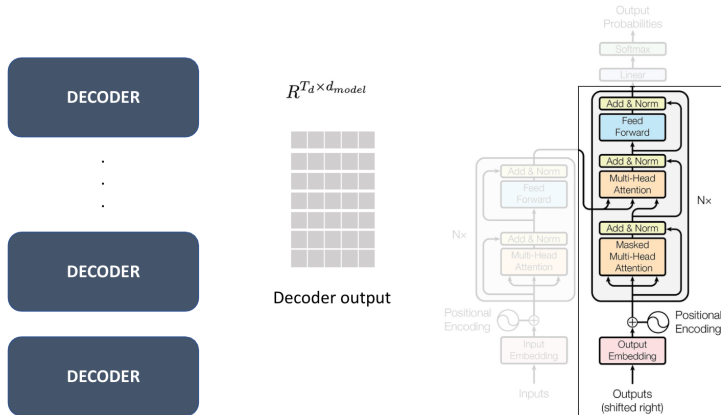
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 106.

- ▶ **Example:** GPT, LLaMA
- ▶ Uses only the decoder block.
- ▶ **Masked self-attention (causal):** Each token can only attend to previous tokens.
- ▶ **Applications:** Language modeling, text generation.

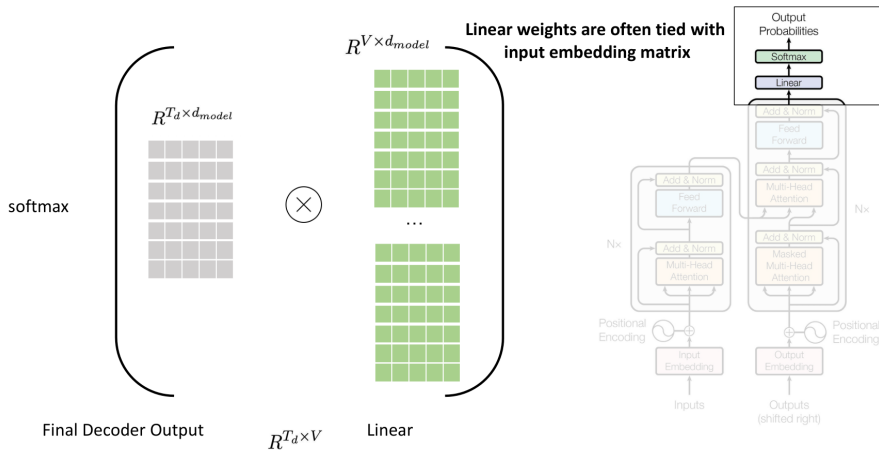
DECODER



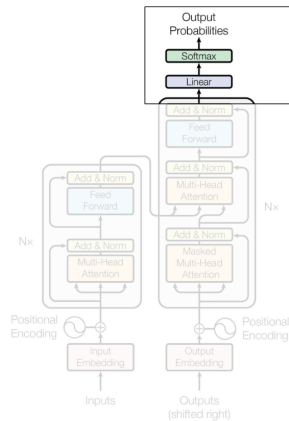
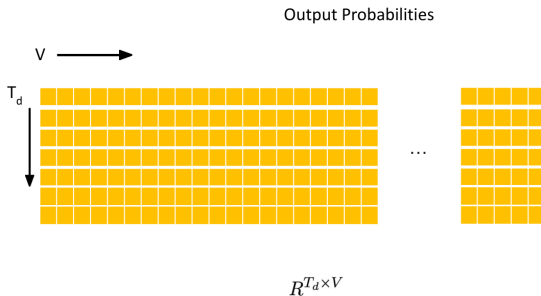
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 107.



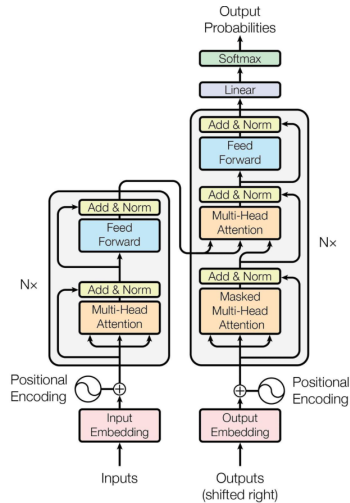
Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 108.



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 109.

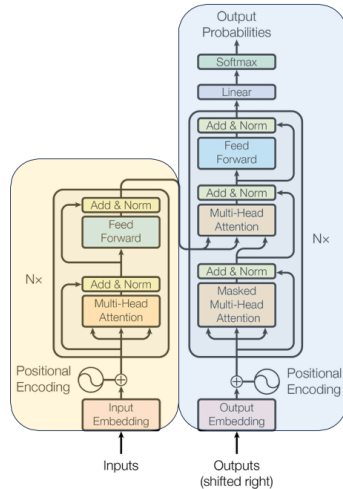


Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 110.



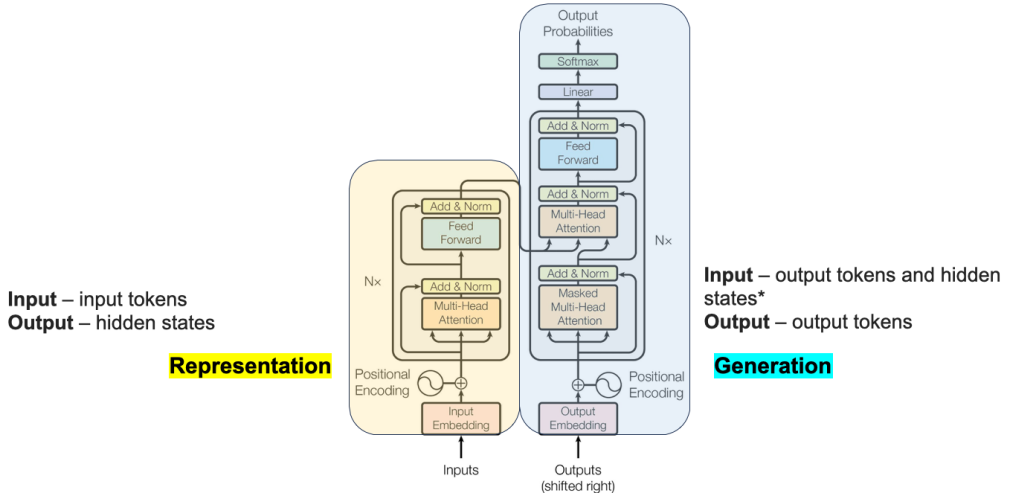
Source: Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1.

Representation



Generation

Architecture diagram: Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1.

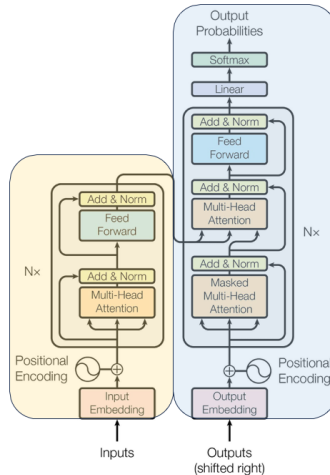


Architecture diagram: Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1.

Input – input tokens
Output – hidden states

Model can see all timesteps

Representation



Input – output tokens and hidden states*

Output – output tokens

Model can only see previous timesteps

Generation

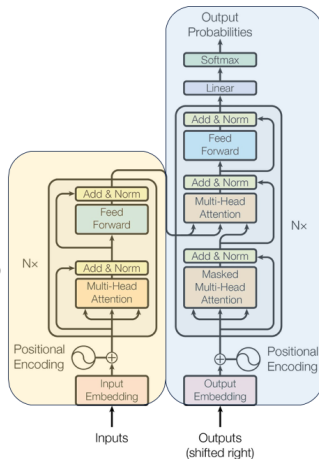
Architecture diagram: Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1.

Input – input tokens
Output – hidden states

Model can see all timesteps

**Does not usually output tokens, so
no inherent auto-regressivity**

Representation



Input – output tokens and hidden states*

Output – output tokens

**Model can only see previous
timesteps**

**Model is auto-regressive with
previous timesteps' outputs**

Generation

Architecture diagram: Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1.

Input – input tokens

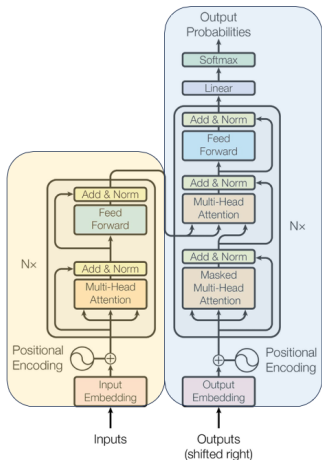
Output – hidden states

Model can see all timesteps

Does not usually output tokens, so no inherent auto-regressivity

Can also be adapted to generate tokens by appending a module that maps hidden state dimensionality to vocab size

Representation



Input – output tokens and hidden states*

Output – output tokens

Model can only see previous timesteps

Model is auto-regressive with previous timesteps' outputs

Can also be adapted to generate hidden states by looking before token outputs

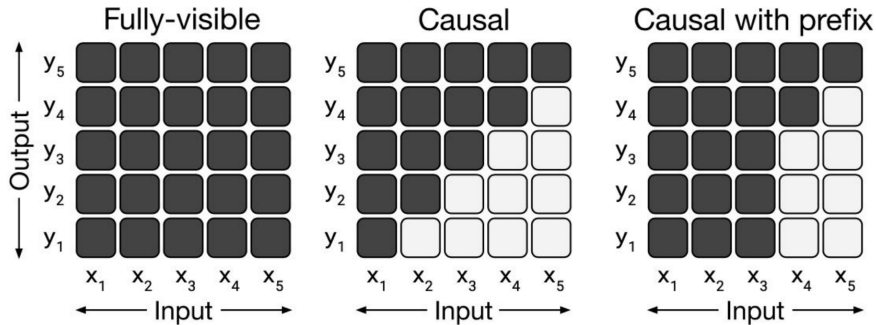
Generation

Architecture diagram: Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1.

- ▶ **Purpose:** Prevents the model from attending to future tokens in the sequence.

- ▶ **Purpose:** Prevents the model from attending to future tokens in the sequence.
- ▶ **Implementation:** Uses a triangular mask in the self-attention mechanism.

- ▶ **Purpose:** Prevents the model from attending to future tokens in the sequence.
- ▶ **Implementation:** Uses a triangular mask in the self-attention mechanism.
- ▶ **Effect:** Ensures that predictions for a token only depend on previous tokens, maintaining the autoregressive property.



Source: Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (T5), JMLR 2020,

Fig. 3.

Target: S1b EOT S2a S2b EOT Target: S1b EOT S2b EOT
Model input: S1a S1b EOT S2a S2b Model input: S1a S1b S2a S2b

Possible attention masks:

	S1	S1	EOT	S2	S2
S1	1	0	0	0	0
S1	1	1	0	0	0
EOT	1	1	1	0	0
S2	1	1	1	1	0
S2	1	1	1	1	1

	S1	S1	S2	S2
S1	1	0	0	0
S1	1	1	0	0
S2	0	0	1	0
S2	0	0	1	1

Left: Causal mask when S1 and S2 can attend to each other

Right: Preventing attention from corss-attending between S1 and S2

Feature	Pre-LN	Post-LN
Stability	✓ More stable	✗ Less stable
Gradient flow	✓ Better	✗ Can explode/vanish
LR warm-up	✓ Can be dropped	✗ Needed at the start
Used in	GPT-3, T5	Original Transformer

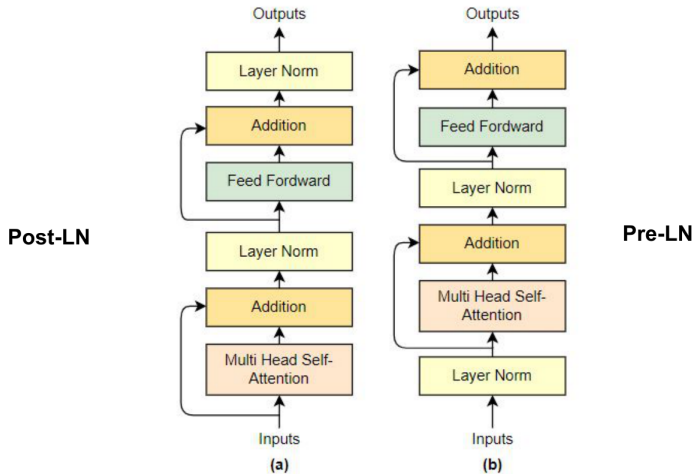
Table 1: Comparison of Pre-LayerNorm and Post-LayerNorm Architectures

Pre-LN: LayerNorm applied **before** sublayers

Post-LN: LayerNorm applied **after** sublayers

With Post-LN the expected gradients near the output layer are large at initialization, which is why a learning-rate warm-up stage is needed; moving the norm inside the residual branch removes that problem (Xiong et al., 2020).

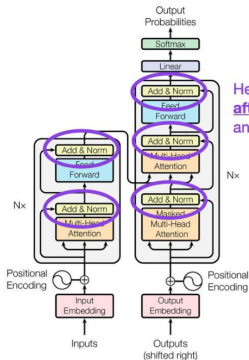
Pre-LayerNorm vs. Post-LayerNorm



Source: after Xiong et al., "On Layer Normalization in the Transformer Architecture," ICML 2020, Fig. 1.

Pre-LayerNorm vs. Post-LayerNorm

Post-LN Transformer

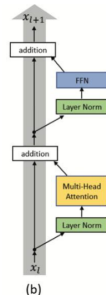
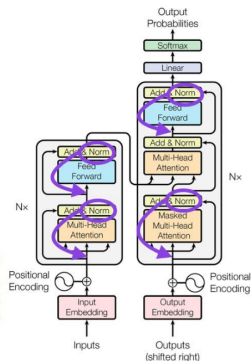


Here, LayerNorms are after attention and after fully connected layers

Better gradients if LayerNorm is placed inside residual connection, before the attention and fully connected layers

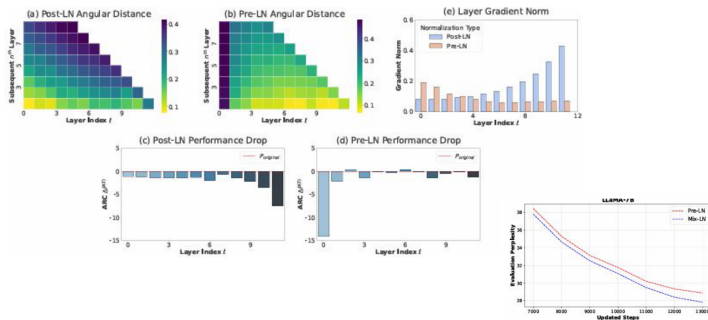
Places the layer normalization between the residual blocks: the expected gradients of the parameters near the output layer are large

Pre-LN Transformer



Source: architecture diagrams from Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1; Pre-LN block at right from Xiong et al., ICML 2020, Fig. 1(b).

Pre-LayerNorm vs. Post-LayerNorm



Post-LN keeps large gradient norms in the deeper layers but attenuates them early on; Pre-LN does the reverse, leaving the deepest layers undertrained.

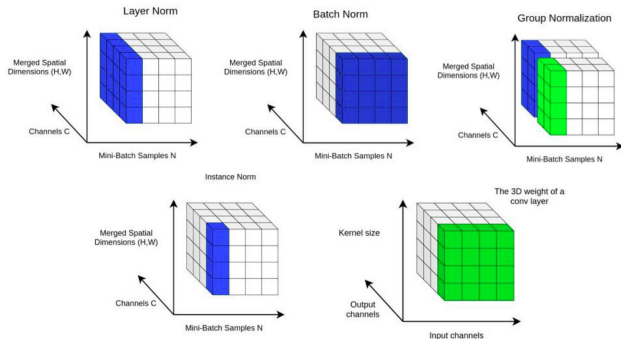
Source: Li, Yin & Liu, "Mix-LN," ICLR 2025, Figs. 3 and 4.

LayerNorm: Normalizes across the feature dimension by subtracting the mean and dividing by the standard deviation.

RMSNorm: Does not subtract the mean; it only re-scales the features by their root mean square (RMS).

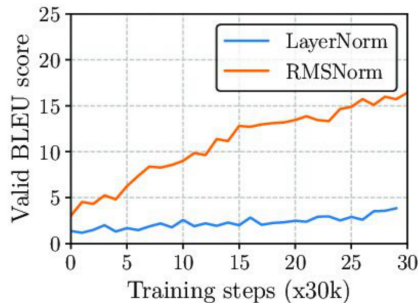
- ▶ Simpler: one statistic per token instead of two, and no learned bias
- ▶ Faster: 7–64% less running time depending on the model (Zhang & Sennrich, 2019)
- ▶ Accuracy is comparable: on WMT14 English-German a Transformer scores 26.8 BLEU with RMSNorm against 26.6 with LayerNorm, at a 7–9% speedup
- ▶ Adopted by LLaMA and the decoder-only models that followed it

What Each Normalization Averages Over



Shaded cells share one mean and variance. RMSNorm pools the same elements as LayerNorm and only drops the mean.

Source: after Wu & He, "Group Normalization," ECCV 2018, Fig. 2.



This is a robustness test: the centre of the weight initialization is shifted to 0.2, under which LayerNorm becomes unstable. With the standard initialization the two are within 0.2 BLEU.

Source: Zhang & Sennrich, "Root Mean Square Layer Normalization," NeurIPS 2019, Fig. 4.

- ▶ **Encoder-only:** BERT, RoBERTa
- ▶ **Decoder-only:** GPT, LLaMA, Falcon
- ▶ **Encoder-Decoder:** T5, BART, mT5

Modifications:

- ▶ **Long-range:** Longformer, BigBird
- ▶ **Efficient:** Linformer, Performer
- ▶ **Sparse:** Sparse Transformer, Reformer

Key Ingredients of BERT:

► Pre-training Corpora:

- English Wikipedia (~2.5B words)
- BooksCorpus (~800M words)

► Pre-training Tasks:

- **MLM** (Masked Language Modeling): Predict randomly masked words in context
- **NSP** (Next Sentence Prediction): Predict if two sentences naturally follow each other

► Model Sizes:

- **BERT-Base**: 110M parameters
- **BERT-Large**: 340M parameters

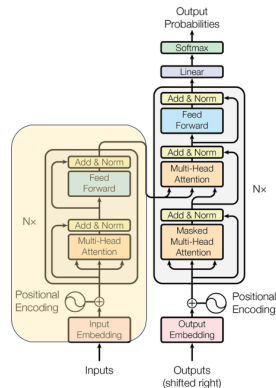


Diagram: Vaswani et al., NeurIPS 2017, Fig. 1.

What is our takeaway from BERT?

► Pre-training tasks can be invented flexibly...

- Effective representations can be derived from a flexible regime of pre-training tasks.

► Different NLP tasks seem to be highly transferable with each other...

- As long as we have effective representations, that forms a general model which can serve as the backbone for many specialized models.

And scaling works!!!

340M was considered large in 2018

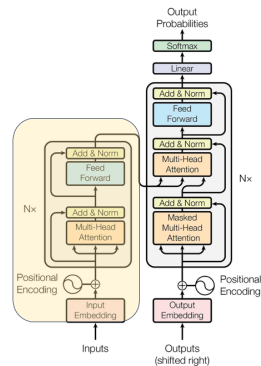


Diagram: Vaswani et al., NeurIPS 2017, Fig. 1.

GPT Pre-Training Corpus:

- BooksCorpus only (over 7,000 unpublished books), no Wikipedia

GPT Pre-Training Tasks:

- Predict the next token, given the previous tokens
- More learning signals than MLM

GPT Pre-Training Results:

- GPT – 117M Params (12 blocks, 768 dims, 12 heads)
- GLUE score of 72.8, up from a previous best of 68.9
- Best results on 9 of the 12 tasks studied

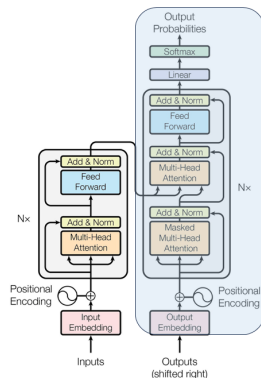


Diagram: Vaswani et al., NeurIPS 2017, Fig. 1.

What is our takeaway from GPT?

► The Effectiveness of Self-Supervised Learning

- Specifically, the model seems to be able to learn from generating the language *itself*, rather than from any specific task we might cook up.

► Language Model as a Knowledge Base

- Specifically, a generatively pretrained model seems to have a decent zero-shot performance on a range of NLP tasks.

And scaling works!!!

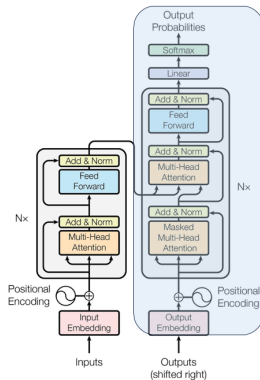


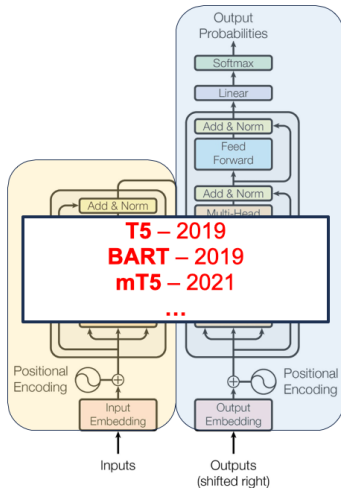
Diagram: Vaswani et al., NeurIPS 2017,

Fig. 1.

The LLM Era – Paradigm Shift in Machine Learning

BERT – 2018
DistilBERT – 2019
RoBERTa – 2019
ALBERT – 2019
ELECTRA – 2020
DeBERTa – 2020
...

Representation



GPT – 2018
GPT-2 – 2019
GPT-3 – 2020
GPT-Neo – 2021
GPT-3.5 (ChatGPT) – 2022
LLaMA – 2023
GPT-4 – 2023
...

Generation

Architecture diagram: Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1.

- ▶ High compute and memory requirements
- ▶ Poor extrapolation capabilities
- ▶ Limitations due to hard-coded positional encoding
- ▶ Quadratic attention cost ($O(n^2)$)
- ▶ Lack of reasoning and grounding

- ▶ Transformers replaced recurrence with attention, which is what made sequence models parallelizable.
- ▶ Self-attention allows contextual understanding.
- ▶ Multi-head and positional encodings add flexibility.
- ▶ Variants serve different tasks (BERT, GPT, T5).
- ▶ Ongoing research improves efficiency, grounding, and capabilities.

These slides have been adapted from:

- ▶ Younes Mourri & Lukasz Kaiser, [Natural Language Processing Specialization. DeepLearning.AI](#)
- ▶ Bhiksha Raj & Rita Singh, [11-785 Introduction to Deep Learning, CMU](#)

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is All You Need. *In NeurIPS*.
2. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *In NAACL*.
3. Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training (GPT). OpenAI.
4. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners (GPT-2). OpenAI.
5. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5). *JMLR*.
6. Wu, Y., & He, K. (2018). Group Normalization. *In ECCV*. arXiv:1803.08494.
7. Zhang, B., & Sennrich, R. (2019). Root Mean Square Layer Normalization. *In NeurIPS*. arXiv:1910.07467.
8. Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T. Y. (2020). On Layer Normalization in the Transformer Architecture. *In ICML*. arXiv:2002.04745.
9. Li, P., Yin, L., & Liu, S. (2025). Mix-LN: Unleashing the Power of Deeper Layers by Combining Pre-LN and Post-LN. *In ICLR*. arXiv:2412.13795.

10. Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding (RoPE). arXiv:2104.09864.
11. Press, O., Smith, N. A., & Lewis, M. (2022). Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation (ALiBi). In *ICLR*. arXiv:2108.12409.
12. Dao, T., Fu, D., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*.
13. Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The Long-Document Transformer. arXiv preprint arXiv:2004.05150.
14. Tay, Y., Dehghani, M., Bahri, D., & Metzler, D. (2020). Efficient Transformers: A Survey. arXiv preprint arXiv:2009.06732.